

Source-Grounded, Multi-Modal Retrieval-Augmented Question Answering: A Survey of Techniques, Systems, and Security Considerations, with SAGE as a Case Study

Srujan Zanjali

Department of Computer Engineering
St. Vincent Pallotti College of Engineering and Technology
Nagpur, India
zanjalsrujan94@gmail.com

Abstract—Retrieval-augmented generation (RAG) has become the dominant practical strategy for grounding large language model outputs in external evidence, but the literature that supports it is fragmented across at least five largely separate threads: core RAG architectures, modality-specific extraction (documents, speech and video, and source code), multi-source “notebook-style” assistants and general-purpose orchestration frameworks, hallucination and confidence estimation, and – increasingly urgent as these systems are given access to arbitrary external content – the security of the retrieval pipeline itself. This survey reviews each thread, synthesises the design trade-offs that recur across it, and argues that a practical multi-modal, source-grounded assistant is best understood as a set of choices along four axes: how broad, low-specificity questions are made retrievable; how citation provenance is preserved per modality; how multiple heterogeneous sources are fairly represented in a combined answer; and how the ingestion and generation steps are hardened against content the system does not control. We use SAGE, a prototype multi-source knowledge engine that ingests websites, PDFs, YouTube videos, and public GitHub repositories, as a running case study that instantiates concrete answers to all four axes – deterministic source-overview chunks, modality-aware citation rendering, diversity-floor reranking, and combined SSRF and indirect-prompt-injection defenses – and compare it qualitatively against a closed consumer assistant (NotebookLM) and general-purpose open-source RAG frameworks (LangChain, LlamaIndex, Haystack). We close with open challenges – calibrated confidence, hybrid lexical-semantic retrieval, and controlled multi-modal benchmarks – that remain unresolved across the surveyed systems.

Index Terms—retrieval-augmented generation, survey, source-grounded question answering, multi-modal retrieval, hallucination, prompt injection, vector databases, document intelligence, video intelligence, code intelligence

I. Introduction

Large language models answer fluently even when they should not. Retrieval-augmented generation (RAG) responds to this by conditioning generation on passages retrieved from an external corpus rather than relying

only on parameters learned at training time, and has been shown to reduce unsupported claims on knowledge-intensive tasks relative to a purely parametric model [1]. In the years since, RAG has moved from a single retrieve-then-generate recipe to a broad family of architectures, and the corpora it grounds against have expanded well beyond short text passages to documents with layout, spoken and video content, and source code repositories. A parallel, more applied literature has produced consumer and developer-facing systems – notebook-style multi-source assistants such as NotebookLM [18], and general-purpose orchestration frameworks such as LangChain [19], LlamaIndex [20], and Haystack [21] – that package these architectural ideas into usable products and libraries.

This survey has two goals. The first is to consolidate five threads of work that are usually read separately – RAG foundations, modality-specific extraction, multi-source assistants and frameworks, hallucination and confidence estimation, and RAG-specific security – into a single account of the design space a builder of a multi-modal, source-grounded assistant actually faces. The second is to ground that account in a concrete system. We use SAGE, a prototype we built that ingests websites, PDFs, YouTube videos, and public GitHub repositories into one retrieval and citation pipeline, as a case study throughout Section IX, not as the survey’s primary contribution but as a way of making the trade-offs discussed in Sections III–VIII concrete and checkable against a working implementation rather than only against claims made in the surveyed papers themselves.

We organise the recurring design space around four questions that, in our reading of the literature, separate a system that merely retrieves passages from one that behaves like a trustworthy source-grounded assistant: (i) how are broad, low-specificity questions (“what is this about”) made retrievable at all, given that dense retrieval

over fine-grained chunks tends to under-serve them; (ii) how is citation provenance preserved in a form appropriate to the source’s modality, since a URL, a page number, a timestamp, and a file/line range are not interchangeable; (iii) when a query spans multiple sources, how is fair representation across sources guaranteed rather than left to whichever source’s chunks happen to score highest; and (iv) given that ingestion fetches content the system does not control, how is the pipeline defended against that content actively working against the system. We return to these four questions in Section IX as the specific criteria against which SAGE, NotebookLM, and generic framework applications are compared.

The rest of this paper is organised as follows. Section II states the survey’s scope. Section III reviews core RAG architectures. Section IV covers modality-specific retrieval and extraction for documents, video and speech, and code. Section V reviews multi-source assistants and orchestration frameworks. Section VI reviews confidence and hallucination in RAG. Section VII reviews security considerations specific to retrieval pipelines. Section VIII reviews hybrid and advanced retrieval strategies. Section IX presents SAGE as a case study and a comparative table. Section X discusses open challenges, and Section XI concludes.

II. Scope and Method

This survey focuses on retrieval-augmented systems that ground answers in externally supplied evidence – as opposed to purely parametric LLM prompting, in-context few-shot learning without retrieval, or fine-tuning approaches to knowledge injection, which we treat as out of scope except where they interact directly with retrieval. Within that focus we prioritise: (a) foundational architectures that later applied work consistently builds on; (b) modality coverage that maps to the four source types a multi-modal assistant like SAGE actually has to handle – web pages, documents, video/speech, and code; (c) systems and frameworks that are either widely deployed (NotebookLM) or widely used as building blocks (LangChain, LlamaIndex, Haystack); and (d) two cross-cutting concerns – faithfulness/hallucination and security – that apply regardless of modality and that we found were often treated as secondary in modality-specific papers despite being first-order concerns for any deployed system. Coverage is necessarily illustrative rather than exhaustive given the pace of publication in this area; we favour frequently-cited foundational work and recent surveys [10]–[12] over an attempt to enumerate every recent variant.

III. Foundations of Retrieval-Augmented Generation

A. From Parametric to Retrieval-Augmented Generation

RAG’s original formulation pairs a pretrained sequence-to-sequence generator with a differentiable retriever over a document index, trained so that the generator learns

to make use of whatever the retriever surfaces [1]. Dense Passage Retrieval (DPR) established the now-standard retriever architecture underlying this and later work: a pair of encoders, one for questions and one for passages, trained so that relevant question-passage pairs are close in embedding space, which outperformed classical sparse (term-matching) retrieval on open-domain QA benchmarks when trained directly on question-passage relevance [2]. Both lines of work depend on the Transformer architecture’s self-attention mechanism for the encoders and generators involved [4], and on sentence-embedding methods such as Sentence-BERT that make large-scale semantic similarity search computationally practical outside of a fine-tuned retrieval-specific model [3].

B. Reader Architectures

Given retrieved passages, how a generator consumes them matters as much as how they were retrieved. Fusion-in-Decoder (FiD) showed that encoding each retrieved passage independently and fusing evidence only in the decoder’s cross-attention lets a generative reader scale to many more retrieved passages than concatenation-based encoding allows, without a quadratic attention cost, and that answer accuracy continues to improve as more passages are retrieved under this scheme [5]. Self-RAG moved a step further by training the model itself to decide, via learned reflection tokens, when retrieval is necessary at all and to critique its own output against retrieved evidence, rather than treating retrieval as an unconditional, fixed pipeline stage that always runs before generation [6]. Read together, these two works mark a shift from “retrieve a fixed k passages, then generate” toward readers that are increasingly aware of how much and which evidence they are actually using.

C. Vector Search Infrastructure

All of the above requires an index that can return the nearest neighbours of a query embedding among potentially millions of candidate vectors quickly enough for interactive use. FAISS demonstrated that approximate nearest-neighbour search scales to billion-vector collections with acceptable recall/latency trade-offs on commodity GPU hardware [8], and remains the substrate under many later, higher-level vector stores. Developer-facing systems such as Chroma build on the same underlying idea but expose it as a collection-oriented API with metadata filtering, designed to be embedded directly inside an application rather than operated as a standalone service [9]. This shift – from “a nearest-neighbour index” to “an application-embeddable vector store with metadata filtering” – is one of the more consequential but under-discussed enablers of the recent wave of RAG applications, since it is what lets an application filter retrieval by, for instance, which knowledgebase or source a chunk came from, rather than only by embedding similarity.

D. Graph-Based and Global-Question RAG

Plain chunk retrieval is well suited to locally-answerable questions but poorly suited to global, corpus-level questions such as “what are the main themes across this entire source”, since no single retrieved chunk is likely to represent the whole. GraphRAG addresses this directly by using an LLM at index time to derive an entity-relationship graph from the source corpus and to pregenerate hierarchical community summaries over that graph, which are then used to answer global, query-focused summarisation questions that conventional RAG underserves [7]. This is, to our knowledge, the most directly comparable prior approach to the broad-question problem that motivates SAGE’s deterministic overview chunks (Section IX): GraphRAG solves the same problem with a substantially heavier index-time pipeline – an LLM call (or many) per corpus to build and summarise the graph – while SAGE trades some of GraphRAG’s summarisation depth for a non-generative, source-derived overview that adds no second hallucination-prone stage at indexing time. We view the two as occupying different points on the same cost/depth trade-off rather than as competitors, and return to this contrast in Section IX.

E. Survey-Level Views of the Field

Broader surveys of RAG as a field consistently identify the same three concerns as dominant in real deployments regardless of the specific architecture used: provenance (can a user trace an answer back to its evidence), updateability (can the corpus change without retraining the model), and modularity (can retrieval, reranking, and generation be swapped independently) [10]. More recent systematic reviews reach similar conclusions while additionally emphasising evaluation methodology as an open problem in its own right – that is, that the field has produced many architectures faster than it has produced agreed-upon ways to measure whether one architecture is actually better than another for a given task [11], [12]. This survey adopts provenance, modularity, and (per Section VI) faithfulness as the organising concerns for the modality-specific and applied sections that follow.

IV. Modality-Specific Retrieval and Understanding

A single-modality RAG system can retrieve fine-grained chunks and stop. A multi-modal assistant must additionally decide, per modality, what the unit of retrieval and citation actually is, since “a paragraph” is not the natural evidence unit for a video or a repository.

A. Documents

For visually structured documents, LayoutLM showed that jointly modelling text content and 2-D layout position improves document understanding tasks relative to text-only models, since meaning in forms, tables, and structured reports depends on spatial arrangement as much as on token sequence [13]. This line of work is most relevant

to scanned or visually rich PDFs; for born-digital PDFs with selectable text, the layout signal is less critical and page-level text extraction is sufficient for reliable retrieval, which is the regime most practical document-QA systems – and the SAGE case study in Section IX – currently target, deferring OCR and layout-aware extraction to future work.

B. Video and Speech

Whisper demonstrated that a single, large, weakly-supervised speech model can achieve robust multilingual transcription without task-specific fine-tuning, which made local, dependency-light transcription practical for downstream applications for the first time at that level of robustness [14]. In practice, systems that ingest video content for QA purposes tend to combine two paths: retrieving existing captions when available, and falling back to a local Whisper-family model only when captions do not exist, since transcription is the more expensive and less accurate of the two options. The retrieval unit for video is naturally a timestamped transcript span rather than a page or paragraph, which has direct consequences for how a citation must be rendered (Section IX).

C. Code

Code introduces retrieval considerations that do not apply to prose: relevance depends on file structure, line locality, and cross-file context, not only on token-level semantic similarity. CodeBERT established that a single model can learn strong joint representations of natural language and code well enough to support natural-language-to-code retrieval tasks [15]. RepoCoder went further, showing that retrieving relevant context from across the repository – not just the current file – substantially improves completion and generation tasks, because a large share of the context a correct completion needs (helper functions, type definitions, project conventions) is not visible in the local file at all [16]. This repository-level-context finding is, in our reading, the single most directly transferable idea from the code-intelligence literature to a general-purpose multi-source assistant: it motivates generating an explicit repository-level overview alongside file-level chunks, rather than treating a repository as an undifferentiated bag of files.

D. Websites

Website ingestion is comparatively under-theorised in the academic literature relative to documents, video, and code, in part because it is treated as a solved crawling/scraping problem rather than a retrieval problem. In practice, two considerations dominate: many modern pages require client-side rendering to expose their real text content, which argues for browser automation (e.g. Playwright [17]) over raw HTTP fetching; and a crawler that follows links on a user-supplied URL is, unless explicitly constrained, both a scope-creep risk (crawling

far beyond the page the user actually cared about) and a security risk, discussed directly in Section VII.

V. Multi-Source and Notebook-Style Assistants

A. Consumer Assistants

Google’s NotebookLM packages several of the above modalities – documents, web pages, and video/audio among them – behind a single consumer-facing, source-grounded assistant, explicitly marketed on the premise that grounding responses in user-supplied sources reduces hallucination relative to an open-domain chatbot [18]. As a closed product, its retrieval, reranking, and citation-rendering internals are not published, which makes it useful as a reference point for what a multi-source assistant can look like to an end user, but not directly reusable as a technical baseline for a research-oriented system – a gap this survey’s case study (Section IX) is positioned to partially fill by describing a comparable, fully open system.

B. Orchestration Frameworks

Where NotebookLM is a fixed product, LangChain [19], LlamaIndex [20], and Haystack [21] are general-purpose libraries: they supply connectors for many data sources, composable indexing and query abstractions, and pipeline components for retrieval, reranking, and generation, from which a developer assembles a specific application. This generality is their central strength and also their central limitation for the purposes of this survey: none of the three frameworks ships a fixed, opinionated answer to the four design questions raised in Section I (broad-question retrievability, per-modality citation format, multi-source fairness, and ingestion-time security) – each is left to the application developer to decide, and in practice is frequently left undecided, which is itself a notable, under-discussed gap between what these frameworks make possible and what they make the default.

VI. Confidence, Faithfulness, and Hallucination in RAG

RAG reduces, but does not eliminate, unsupported generation: a retriever can return irrelevant or only partially relevant passages, and a generator can still produce claims unsupported by whatever was retrieved. A substantial literature studies this directly. Comprehensive surveys of hallucination in large language models organise the failure modes into taxonomies spanning input-conflicting, context-conflicting, and fact-conflicting hallucination, and catalogue detection and mitigation techniques operating at the data, training, and inference stages respectively [22]. Detection techniques that are particularly relevant to a retrieval-augmented setting include sampling-based, black-box consistency checks that flag likely hallucinations by testing whether repeated generations from the same model remain internally consistent, without requiring access to model internals or a labelled dataset – a property that makes such techniques attractive for a system, like

most practical RAG deployments, built on a hosted, closed LLM API rather than a locally fine-tuned model.

Confidence estimation in deployed RAG systems tends to be considerably simpler than this literature: rather than a calibrated posterior probability, most practical systems – including the SAGE case study in Section IX – compute an interpretable heuristic from retrieval and reranking signals (best score, evidence agreement, citation count) and use it to gate whether the system answers at all, or to display a confidence label to the end user. This is a defensible engineering compromise given the cost of true calibration, but it also means the confidence values surfaced by most deployed assistants should be read as operational guidance rather than as a rigorous faithfulness guarantee – a distinction we found was rarely made explicit in product-facing documentation, though it is well understood in the research literature [22].

VII. Security Considerations in RAG Pipelines

A RAG system that retrieves external content before generating a response has an attack surface that a closed, purely parametric chatbot does not, and this surface grows directly with the number of external source types the system is willing to ingest.

A. Indirect Prompt Injection

Greshake et al. formalised indirect prompt injection: an attacker does not interact with the LLM-integrated application directly, but instead places adversarial instructions inside content – a web page, a document, a retrieved search result – that the application will later fetch and place into the model’s context, at which point the model may treat those instructions as if they came from the application’s own system prompt or the user [23]. The paper’s threat taxonomy (data exfiltration, worming across chained agents, ecosystem contamination, and unauthorised tool or API invocation) applies directly to any multi-source assistant, since every ingested website, PDF, transcript, or repository file is, from the model’s perspective, content the system chose to trust enough to place in context. This risk is now formally ranked as the top item in the OWASP Top 10 for LLM Applications [24], reflecting a broader industry consensus that it is not a theoretical concern but the most commonly exploited weakness in deployed LLM-integrated applications.

B. Server-Side Request Forgery in Ingestion

A second, less frequently discussed risk is specific to systems that accept a user-supplied URL as an ingestion target: unless every request the system makes on that URL’s behalf – including automatic HTTP redirects and, for browser-automated crawlers, in-page navigations triggered after the initial request – is validated against private, loopback, link-local, and cloud-metadata address ranges, the ingestion endpoint becomes a general-purpose SSRF primitive that can be used to probe or reach infrastructure

the operator did not intend to expose. This risk is well documented in general web-application security guidance but is, in our reading of the RAG-specific literature, rarely discussed alongside indirect prompt injection despite affecting the same class of system for the same underlying reason: both stem from treating ingestion-time content as fully trusted once a single, initial check has passed.

C. Implications for System Design

Taken together, these two risks argue for security controls that are structural rather than input-filtering-based: revalidating every network hop rather than only the user-supplied URL, and delimiting retrieved content from instructions at the prompt level rather than attempting to detect and strip injected instructions after the fact, since detection-based defenses against injected instructions are themselves an active and unsettled research problem [23], [24]. Section IX describes how the SAGE case study implements both controls concretely.

VIII. Hybrid and Advanced Retrieval Strategies

Dense, embedding-based retrieval and classical sparse, term-based retrieval (e.g. BM25) fail on complementary classes of query: dense retrieval generalises well across paraphrase and synonymy but can miss passages that share no semantic neighbourhood with the query despite containing an exact match for a code identifier, error string, or proper noun, while sparse retrieval is precise on exact terms but blind to paraphrase. A substantial and consistent empirical finding across the retrieval literature is that combining the two – for example via Reciprocal Rank Fusion (RRF), which combines multiple rankings by summing reciprocal-rank scores rather than requiring the rankings’ underlying scores to be comparable – reliably outperforms either method used alone, and does so without requiring a trained fusion model [25]. This finding predates the current wave of LLM-centric RAG systems by over a decade, but is, in our assessment, under-adopted in practice: many deployed RAG pipelines, including single-vector-store architectures like the SAGE case study discussed next, currently rely on dense retrieval alone or on a fixed linear blend of dense and lexical scores rather than a full hybrid pipeline, which we identify in Section X as one of the more immediately actionable gaps across the systems surveyed here.

IX. Case Study: SAGE

We use SAGE, a prototype we built and validated, to make the design space of Sections III–VIII concrete. SAGE ingests four source types – public websites, born-digital PDFs, YouTube videos, and public GitHub repositories – into a shared chunk-and-metadata representation: one knowledgebase per source, a deterministic overview chunk generated from source statistics and excerpts (not a second LLM call) followed by detailed chunks, and provenance metadata specific to the modality (URL and chunk index

TABLE I
The four organising design questions, answered by SAGE, NotebookLM [18], and a representative application built on a general-purpose framework [19]–[21].

Question	SAGE	NotebookLM	Framework app
Broad-question retrievability	Deterministic overview chunk per source	Not disclosed	Left to developer
Per-modality citation	URL, page, timestamp, or file/line (fixed per modality)	Citations shown; internals undisclosed	Left to developer
Multi-source fairness	Diversity-floor rerank (Alg. guaranteed)	Not disclosed	Left to developer
Ingestion & generation security	Explicit per-hop SSRF check + injection-guarded prompting	Not disclosed	Left to developer

for websites, page number for PDFs, timestamp span for video, file path and line range for GitHub). Unlike GraphRAG’s LLM-built entity graph and community summaries [7], SAGE’s overview chunk is compiled deterministically from source statistics and excerpts, trading summarisation depth for a non-generative pipeline stage that cannot itself introduce new, unsupported claims – a design choice appropriate to a system whose corpora are ingested from arbitrary, untrusted public sources rather than a curated private dataset. Retrieval embeds chunks with a compact sentence encoder [26], stores vectors in Chroma [9], and stores relational metadata and lifecycle state in Supabase PostgreSQL [27]. At query time, SAGE optionally rewrites short or ambiguous questions using recent conversation turns, retrieves and reranks candidate chunks with a linear blend of semantic and lexical scores, and – for questions that combine more than one selected source – applies a diversity-floor step that reserves each source’s best-scoring chunk before filling remaining slots by pure score, guaranteeing every selected source is represented in the final citation set rather than only the highest-scoring one. Generation runs in a Grounded Mode that refuses when retrieval evidence is weak, or an Exploratory Mode that presents source-backed content before any broader explanation; both modes wrap retrieved content in explicit, delimited tags with an instruction-injection guard directly motivated by [23], and ingestion validates every fetched URL and redirect hop against private/loopback/link-local/cloud-metadata address ranges.

Table I maps SAGE’s concrete design choices onto the four organising questions from Section I, alongside NotebookLM and a representative application built on a general-purpose framework.

The pattern in Table I is the survey’s central observation about the current state of the field: none of these four questions is architecturally hard to answer well, but only a fixed, opinionated system – whether a closed consumer product or an open prototype like SAGE – actually answers all four by default, while a general-purpose framework leaves all four to whoever assembles the final application, which in our reading of deployed framework-based projects is inconsistently done at best.

X. Open Challenges and Future Directions

Four gaps recur across the systems and literature surveyed here, and remain open in the SAGE case study as much as elsewhere. First, calibrated confidence: nearly every practical system, SAGE included, substitutes an interpretable heuristic for a calibrated faithfulness estimate, and closing this gap requires either adopting sampling-based hallucination-detection techniques from the literature [22] at inference time, at some latency and cost premium, or investing in a controlled, annotated evaluation set specific to the deployment domain. Second, hybrid retrieval: despite over a decade of evidence that combining dense and sparse retrieval outperforms either alone [25], single-vector-store architectures remain common, largely because they are simpler to operate; the actual engineering cost of adding a sparse index alongside an existing dense one is modest relative to the precision gained on exact-match queries. Third, security as a first-class evaluation criterion: indirect prompt injection and ingestion-time SSRF are both well documented as risks [23], [24], but we found no widely-adopted benchmark that scores a RAG system’s resistance to either, which leaves builders to implement structural defenses (as SAGE does) without a standard way to verify their sufficiency. Fourth, cross-modal evaluation: existing RAG benchmarks are overwhelmingly single-modality (document QA or code completion, rarely both, and almost never alongside video), which makes it difficult to compare multi-modal assistants like SAGE and NotebookLM on any common, reproducible ground rather than on vendor-reported or informally validated results.

XI. Conclusion

This survey reviewed retrieval-augmented generation from its core architectures through modality-specific extraction, multi-source assistants and orchestration frameworks, confidence and hallucination, and the security of the retrieval pipeline itself, and organised the resulting design space around four questions: broad-question retrievability, per-modality citation provenance, multi-source fairness, and ingestion/generation security. Using SAGE as a case study showed that all four questions admit concrete, implementable answers – deterministic overview chunks, modality-aware citation metadata, diversity-floor reranking, and combined SSRF and injection defenses – at a cost that is modest relative to building any RAG pipeline

at all, which suggests the gap between a merely functional RAG system and a trustworthy, source-grounded one is primarily a design-discipline problem rather than a research problem requiring new architectures. Calibrated confidence, hybrid retrieval, security benchmarking, and cross-modal evaluation remain the open problems most likely to determine whether the next generation of these systems closes that gap further.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [4] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [5] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” in *Proc. 16th Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2021, pp. 874–880.
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- [7] D. Edge et al., “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [8] J. Johnson, M. Douze, and H. Jégou, “Billion-scale Similarity Search with GPUs,” *arXiv preprint arXiv:1702.08734*, 2017.
- [9] Chroma, “Chroma Documentation,” 2026. [Online]. Available: <https://docs.trychroma.com>. Accessed: Jul. 10, 2026.
- [10] Y. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [11] Anonymous, “A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics, and Challenges,” *arXiv preprint arXiv:2508.06401*, 2025.
- [12] Anonymous, “Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey,” *arXiv preprint arXiv:2504.14891*, 2025.
- [13] Y. Xu et al., “LayoutLM: Pre-training of Text and Layout for Document Image Understanding,” in *Proc. ACM SIGKDD*, 2020, pp. 1192–1200.
- [14] A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. ICML*, vol. 202, 2023, pp. 28492–28518.
- [15] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of EMNLP*, 2020, pp. 1536–1547.
- [16] F. Zhang et al., “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” in *Proc. EMNLP*, 2023, pp. 2471–2484.
- [17] Microsoft, “Playwright Documentation,” 2026. [Online]. Available: <https://playwright.dev/>. Accessed: Jul. 10, 2026.
- [18] Google, “NotebookLM: AI Research Tool and Thinking Partner,” 2026. [Online]. Available: <https://notebooklm.google/>. Accessed: Jul. 10, 2026.
- [19] LangChain, “LangChain Documentation,” 2026. [Online]. Available: <https://www.langchain.com/>. Accessed: Jul. 10, 2026.
- [20] LlamaIndex, “LlamaIndex Documentation,” 2026. [Online]. Available: <https://docs.llamaindex.ai/>. Accessed: Jul. 10, 2026.
- [21] deepset, “Haystack: Open-Source LLM Orchestration Framework,” 2026. [Online]. Available: <https://github.com/deepset-ai/haystack>. Accessed: Jul. 10, 2026.

- [22] L. Huang et al., “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions,” *ACM Transactions on Information Systems*, 2024.
- [23] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [24] OWASP Gen AI Security Project, “LLM01:2025 Prompt Injection,” *OWASP Top 10 for LLM Applications*, 2025. [Online]. Available: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>. Accessed: Jul. 10, 2026.
- [25] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759.
- [26] Sentence Transformers, “all-MiniLM-L6-v2 Model Card,” 2026. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. Accessed: Jul. 10, 2026.
- [27] Supabase, “Database Overview,” 2026. [Online]. Available: <https://supabase.com/docs/guides/database/overview>. Accessed: Jul. 10, 2026.